МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное образовательное бюджетное учреждение высшего образования
«ФИНАНСОВЫЙ УНИВЕРСИТЕТ ПРИ ПРАВИТЕЛЬСТВЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»
(Финансовый университет)

Факультет информационных технологий и анализа больших данных
Кафедра информационных технологий

ОТЧЁТ

о выполнении практической работы № 2

по дисциплине «Кросс-платформенная разработка»

Тема: «Знакомство с инструментальными средствами разработки кросс-платформенных программных продуктов»

Вариант 7

Выполнил:
студент группы ТРПО 23-1
Манохин Ярослав Алексеевич
Дата выполнения: 27.09.2026

Проверил:
_____

Москва — 2026

Введение

Цель работы — сформировать системное представление о современных инструментальных средствах разработки кросс-платформенных программных продуктов и получить первичный опыт их применения.

Вариант 7 (последняя цифра номера зачётной книжки 230597): сравнение React Native и Ionic + Capacitor для контентного приложения новостного портала (iOS, Android, веб) при приоритете скорости разработки.

Способ проверки сведений. Актуальные версии и даты релизов получены 27.09.2026 из первоисточников скриптом `tools/fetch_versions.py`: страницы Releases на GitHub (через GitHub API), официальные API и каталоги загрузки (python.org, nodejs.org, метаданные релизов .NET, архив релизов Flutter, download.qt.io, реестр npm). Результат с адресом источника для каждой записи сохранён в `data/tool_versions.json`. Сведения из учебной литературы для версий не использовались.

Материалы работы размещены в Git-репозитории `crossplatform-practice-02`: отчёт — в каталоге `report/`, скрипты — в `tools/`, собранные данные — в `data/`.

1. Задание 1. Классификация инструментальных средств

В таблицу 1 включено 16 инструментов, покрывающих все семь категорий таблицы 1 методических указаний. Лицензии указаны по данным GitHub API и официальных страниц лицензирования, примеры приложений — по официальным витринам (showcase) и репозиториям разработчиков.

Таблица 1 — Классификация инструментальных средств кросс-платформенной разработки (по состоянию на 27.09.2026)

№	Категория	Инструмент	Разработчик	Язык	Целевые платформы	Механизм переносимости	Лицензия	Актуальная версия (дата релиза)	Пример приложения
1	Интерпретируемые языки и среды выполнения	Python	Python Software Foundation	Python	Windows, macOS, Linux, BSD и др.	интерпретатор CPython и стандартная библиотека собраны для каждой ОС; программа переносится исходным текстом	PSF-2.0	3.14.7 (05.08.2026)	Instagram (серверная часть), клиент Dropbox
2	Интерпретируемые языки и среды выполнения	Node.js	OpenJS Foundation	JavaScript, TypeScript	Windows, macOS, Linux	движок V8 и библиотека libuv скрывают различия ОС (файлы, сеть, процессы)	MIT	24.21.0 LTS «Krypton» (07.09.2026); 26.10.0 Current (21.09.2026)	npm, серверная часть Netflix и PayPal
3	Виртуальные машины и байт-код	.NET (CLR)	Microsoft, .NET Foundation	C#, F#, Visual Basic	Windows, macOS, Linux; Android и iOS — через .NET MAUI	компиляция в байт-код CIL, исполнение в CLR с JIT- или AOT-компиляцией	MIT	10.0.12 (LTS) (08.09.2026)	Stack Overflow, Bing
4	Виртуальные машины и байт-код	WebAssembly (среда Wasmtime)	W3C WebAssembly WG; Bytecode Alliance	C, C++, Rust, Go, C# и др. → Wasm	браузеры; вне браузера — Windows, macOS, Linux	переносимый байт-код, исполняемый ВМ браузера или автономной средой выполнения	спецификация W3C; Apache-2.0	спецификация Wasm 3.0 (17.09.2025); Wasmtime 49.0.1 (24.09.2026)	Figma, Google Earth (веб)
5	Переносимый исходный код	CMake	Kitware	C, C++ (язык сценариев CMake)	Windows, macOS, Linux, Android, iOS, встраиваемые системы	единое описание сборки → Makefile, Ninja, Visual Studio, Xcode; компиляция под каждую платформу	BSD-3-Clause	4.4.3 (25.08.2026)	сборка Qt 6, LLVM, KDE
6	Кросс-платформенные UI-фреймворки (десктоп)	Qt 6	The Qt Company, Qt Project	C++, QML	Windows, macOS, Linux, Android, iOS, встраиваемые; ОС «Аврора»	единый API и собственная отрисовка (Qt Widgets, Qt Quick); кросс-компиляция	LGPL-3.0 / GPL-3.0 / коммерческая	6.11.2 (18.08.2026)	Telegram Desktop, KDE Plasma
7	Кросс-платформенные UI-фреймворки (десктоп)	Avalonia UI	AvaloniaUI OÜ	C#, XAML	Windows, macOS, Linux, iOS, Android, WebAssembly	собственная отрисовка (Skia) поверх .NET — одинаковый вид на всех ОС	MIT	12.1.3 (22.09.2026)	JetBrains dotMemory и dotTrace, Git Credential Manager
8	Веб-технологии и WebView-обёртки	Electron	OpenJS Foundation (создан GitHub)	JavaScript, TypeScript, HTML/CSS	Windows, macOS, Linux	в каждый дистрибутив встроены Chromium и Node.js	MIT	44.4.5 (23.09.2026)	VS Code, Discord, Slack

№	Категория	Инструмент	Разработчик	Язык	Целевые платформы	Механизм переносимости	Лицензия	Актуальная версия (дата релиза)	Пример приложения
9	Веб-технологии и WebView-обёртки	Tauri 2	Tauri Programme (The Commons Conservancy)	Rust + веб-интерфейс (JS/TS)	Windows, macOS, Linux, Android, iOS	системный WebView (WebView2, WKWebView, WebKitGTK) и ядро на Rust	Apache-2.0 / MIT	2.12.0 (26.09.2026)	GitButler
10	Кросс-платформенные мобильные фреймворки	Flutter	Google	Dart	iOS, Android, Web, Windows, macOS, Linux; ОС «Аврора» (порт ОМП)	собственный движок отрисовки Impeller (в вебе — Skia), AOT-компиляция Dart	BSD-3-Clause	3.47.5 (18.09.2026)	Google Pay, NotebookLM
11	Кросс-платформенные мобильные фреймворки	React Native + Expo	Meta; Expo	JavaScript, TypeScript (React)	iOS, Android; веб, Windows, macOS — внешние платформы	нативные компоненты ОС; новая архитектура (Fabric, TurboModules, JSI), движок Hermes	MIT	React Native 0.87.1 (26.08.2026); Expo SDK 57 (24.09.2026)	Facebook, Instagram, Microsoft Office
12	Кросс-платформенные мобильные фреймворки	Kotlin Multiplatform / Compose Multiplatform	JetBrains	Kotlin	Android, iOS, десктоп (JVM), веб (Wasm)	общая логика на Kotlin (JVM-байт-код, Kotlin/Native, JS/Wasm) и общий UI Compose (Skia)	Apache-2.0	Kotlin 2.4.20 (07.09.2026); Compose MP 1.12.1 (22.09.2026)	Netflix, Duolingo, Booking.com
13	Кросс-платформенные мобильные фреймворки	.NET MAUI	Microsoft	C#, XAML	Android, iOS, Windows, macOS (Mac Catalyst)	единый проект; нативные элементы управления ОС через обработчики (handlers)	MIT	10.0.110 (22.09.2026)	эталонное приложение .NET Podcasts (Microsoft)
14	Кросс-платформенные мобильные фреймворки	Ionic + Capacitor	Ionic (OutSystems)	JavaScript, TypeScript (Angular, React, Vue)	iOS, Android, Web (PWA)	веб-интерфейс (Web Components) в системном WebView и нативные плагины Capacitor	MIT	Ionic 9.0.5 (23.09.2026); Capacitor 8.5.2 (11.09.2026)	приложения Southwest Airlines, NHS, Sanvello
15	Виртуализация и контейнеризация	Docker Engine	Docker, Inc.; проект Moby	любой (образы); реализован на Go	Linux; Windows и macOS — через Docker Desktop	контейнер: образ со всеми зависимостями запускается одинаково на любом хосте	Apache-2.0	29.8.1 (15.09.2026)	среды сборки CI: GitHub Actions, GitLab CI
16	Виртуализация и контейнеризация	WSL 2	Microsoft	любой (программы Linux)	Windows 10, 11	лёгкая ВМ Hyper-V с ядром Linux: сборка и тестирование Linux-версий на Windows	MIT	2.7.14 (11.09.2026)	бэкенд Docker Desktop, VS Code Remote — WSL

Наблюдения. Сверка с первоисточниками показала, что часть сведений методических указаний и учебной литературы уже устарела:

– React Native: в указаниях — версия 0.82 (10.2025, новая архитектура стала единственной); актуальна 0.87.1, с версии 0.84 движок Hermes V1 используется по умолчанию [9];

– Compose Multiplatform: в указаниях — 1.8 (05.2025); актуальна 1.12.1;

– Flutter 3.47.5: с версии 3.47 движок Impeller включён по умолчанию и на десктопе (Windows, macOS, Linux), в вебе используется Skia [10];

– Tauri: стабильная линия — 2.12.0, параллельно выпускаются альфа-версии Tauri 3.0 (в таблицу не включены как предварительные); .NET 11 находится на стадии RC, поэтому указан .NET 10 LTS;

– Node.js: актуальная версия — 24.21.0 LTS «Krypton», установленная на рабочем месте 24.14.1 относится к той же линии, но отстаёт на несколько исправлений — её следует обновить;

– в репозитории flutter/samples больше нет примеров provider_shopper и counter из таблицы 5 указаний (см. задание 3).

2. Задание 2. Подготовка рабочего окружения

Visual Studio Code, Git и Node.js LTS установлены на рабочем месте; версии зафиксированы на рисунке 1. Команда `code` в PATH указывает на удалённый редактор Cursor, поэтому VS Code вызван по полному пути. Облачные среды (DartPad, Expo Snack, Codespaces) не понадобились: все инструменты работают локально.

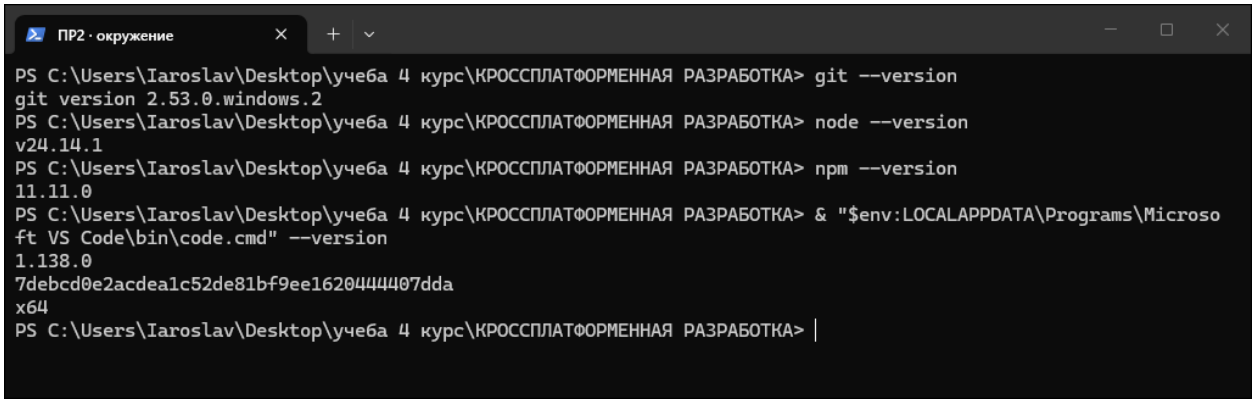


Рисунок 1 — Версии Git, Node.js, npm и Visual Studio Code

Репозиторий `crossplatform-practice-02` создан с файлом `README.md` (тема работы, ФИО, группа), затем клонирован командой `git clone`, выполнены пробный коммит и отправка на сервер `git push` (рисунок 2). По решению обучающегося репозиторий не публиковался на GitHub, GitLab или GitVerse: роль сервера играет локальный bare-репозиторий `_git-server/crossplatform-practice-02.git`. Команды `clone`, `commit` и `push` работают с ним так же, как с удалённой платформой. Для публикации достаточно создать пустой публичный репозиторий на выбранной платформе и выполнить `git remote set-url origin <адрес>` и `git push -u origin main`.

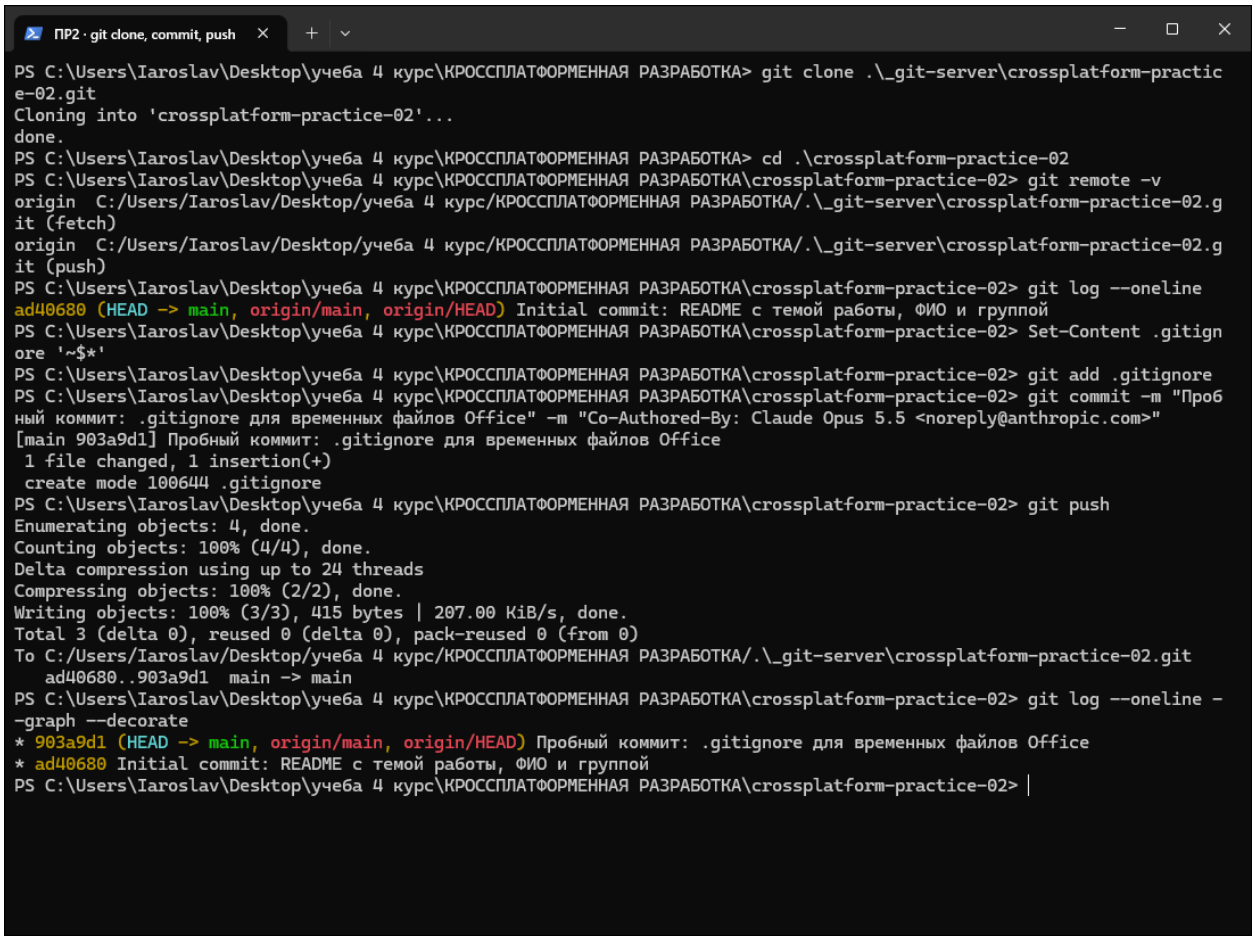


Рисунок 2 — Клонирование, пробный коммит, отправка на сервер и история коммитов

3. Задание 3. Устройство кросс-платформенного проекта

Выбор проекта. Рекомендованные в таблице 5 примеры `counter` и `provider_shopper` отсутствуют в актуальном репозитории flutter/samples (проверено по коммиту 8a4cf1d от 14.09.2026 командой `git ls-tree`).

Поэтому выбран официальный пример того же репозитория — `platform_channels`. Он удовлетворяет критериям указаний: небольшой (10 файлов общего кода), содержит платформенные каталоги `android/` и `ios/` и файл конфигурации `pubspec.yaml`, а главное — наглядно показывает границу между общим и платформенным кодом.

Проект скачан без истории, причём только нужный каталог (листинг 1); запуск и сборка не выполнялись.

Листинг 1 — Загрузка примера

```
git clone --depth 1 --filter=blob:none --sparse https://github.com/flutter/samples.git
flutter-samples
cd flutter-samples
git sparse-checkout set platform_channels
```

Дерево каталогов в Visual Studio Code — на рисунках 3–5; назначение каталогов и файлов — в таблице 2.

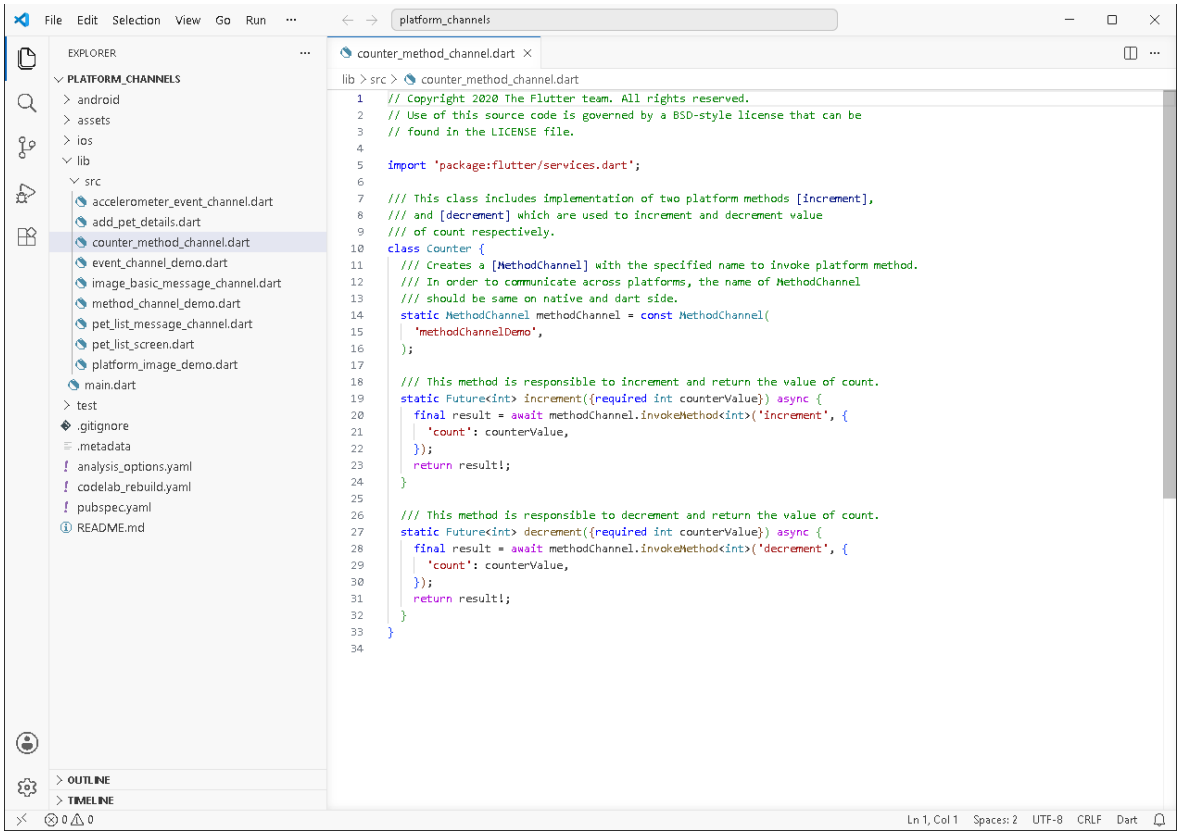


Рисунок 3 — Общий код lib/ (Dart): обёртка MethodChannel

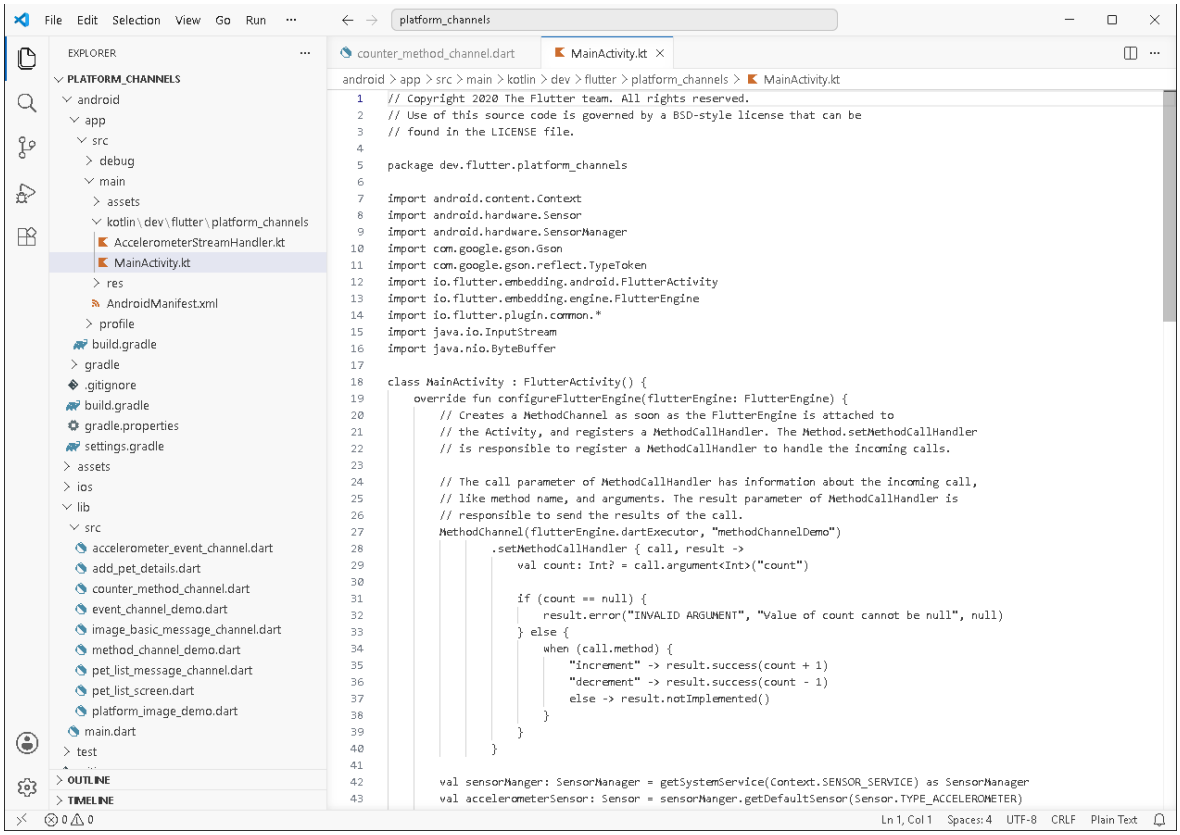


Рисунок 4 — Платформенный каталог android/: MainActivity.kt

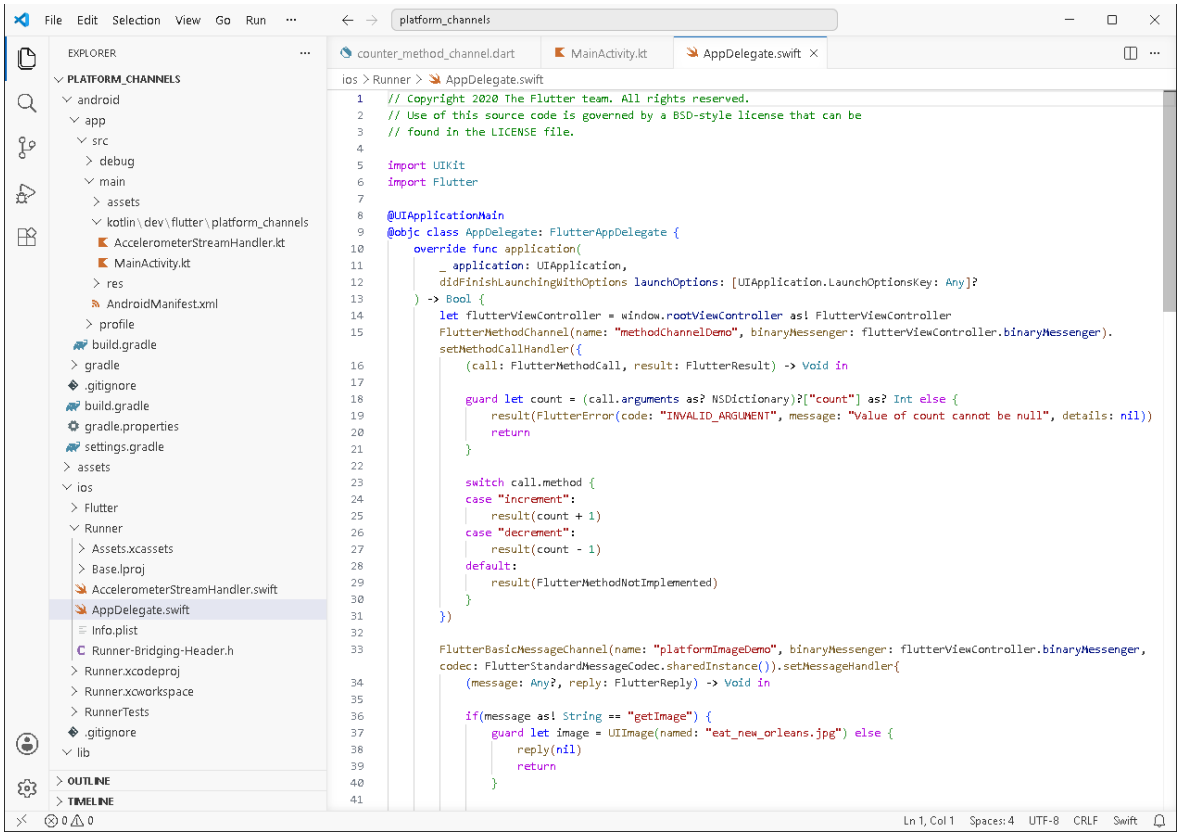


Рисунок 5 — Платформенный каталог ios/: AppDelegate.swift

Таблица 2 — Структура кросс-платформенного проекта platform_channels

Каталог / файл	Назначение	Относится к
`lib/main.dart`	точка входа: `MaterialApp.router`, маршруты `GoRouter`, главный экран со списком демонстраций	общей кодовой базе
`lib/src/*_demo.dart`, `pet_list_screen.dart`, `add_pet_details.dart`	экраны (виджеты) демонстраций	общей кодовой базе
`lib/src/counter_method_channel.dart`	`MethodChannel('methodChannelDemo')`: вызов методов `increment` / `decrement` платформы	общей базе (обращение к платформе)
`lib/src/accelerometer_event_channel.dart`	`EventChannel('eventChannelDemo')`: поток показаний акселерометра	общей базе (обращение к платформе)
`lib/src/image_basic_message_channel.dart`	`BasicMessageChannel('platformImageDemo')`, `StandardMessageCodec`: изображение из нативных ресурсов	общей базе (обращение к платформе)
`lib/src/pet_list_message_channel.dart`	каналы `jsonMessageCodecDemo` (JSON) и `binaryCodecDemo` (Binary)	общей базе (обращение к платформе)
`test/`	виджет-тесты; каналы подменяются через `setMockMethodCallHandler` / `setMockMessageHandler`, устройство не нужно	общей кодовой базе
`pubspec.yaml`	манифест: имя, версия, SDK Dart `^3.9.0`, зависимости `flutter`, `cupertino_icons`, `go_router`, ассеты	общей базе (конфигурация и зависимости)
`analysis_options.yaml`, `.metadata`, `README.md`, `codelab_rebuild.yaml`	правила анализатора, ревизия Flutter и список платформ (android, ios), описание, сценарий пересборки codelab	общей базе (служебные)
`assets/eat_new_orleans.jpg`	общий ресурс Flutter	общей кодовой базе
`android/app/src/main/kotlin/…/MainActivity.kt`	`FlutterActivity`: регистрирует обработчики всех каналов на Kotlin (`setMethodCallHandler`, `setStreamHandler`, `setMessageHandler`), JSON через Gson	Android
`android/…/AccelerometerStreamHandler.kt`	`EventChannel.StreamHandler` + `SensorEventListener`: `SensorManager` → `eventSink`	Android
`android/app/build.gradle`, `android/build.gradle`, `settings.gradle`, `gradle.properties`, `gradle/wrapper/`	сборка Gradle: плагин Flutter, `applicationId`, `minSdk` / `targetSdk` из Flutter, зависимость Gson	Android (сборка)

Каталог / файл	Назначение	Относится к
android/app/src/main/ AndroidManifest.xml, src/debug, src/profile	манифест приложения для сборок release / debug / profile	Android
android/app/src/main/ res/, src/main/assets/	значки, тема экрана запуска; копия изображения для assets.open	Android
ios/Runner/ AppDelegate.swift	FlutterAppDelegate: обработчики каналов на Swift (FlutterMethodChannel и др.)	iOS
ios/Runner/ AccelerometerStreamHandler.swift	FlutterStreamHandler на CoreMotion (CMMotionManager)	iOS
ios/Runner.xcodeproj, Runner.xcworkspace, Flutter/*.xcconfig, Info.plist, *.storyboard	проект Xcode, конфигурации сборки, свойства и экран запуска приложения	iOS (сборка)
ios/Runner/ Assets.xcassets/, ios/RunnerTests/	значки и изображение для UIImage(named:); нативные тесты iOS	iOS

Обращения к платформозависимым возможностям. В общем коде найдено четыре таких места — все через каналы платформы (рисунок 6):

1) Counter.increment / decrement (counter_method_channel.dart) вызывают methodChannel.invokeMethod('increment', {'count': …}); на Android вызов обрабатывает MethodChannel(…, "methodChannelDemo").setMethodCallHandler в MainActivity.kt, на iOS — FlutterMethodChannel(name: "methodChannelDemo", …) в AppDelegate.swift;

2) Accelerometer.readings (accelerometer_event_channel.dart) подписывается на EventChannel('eventChannelDemo').receiveBroadcastStream(); данные поставляет датчик ОС: SensorManager на Android, CMMotionManager (CoreMotion) на iOS;

3) PlatformImageFetcher.getImage (image_basic_message_channel.dart) запрашивает изображение из нативных ресурсов: assets.open(…) на Android, UIImage(named:) на iOS;

4) `PetListMessageChannel` и `PetListScreen` обмениваются данными каналами с кодеками JSON, Binary и String; список питомцев хранится в нативной части.

Условных проверок ОС (`Platform.isAndroid`, `kIsWeb`, `defaultTargetPlatform`) в `lib/` нет — это проверено поиском по коду. Различия платформ полностью вынесены за каналы, а тесты подменяют каналы заглушками и выполняются без устройства.

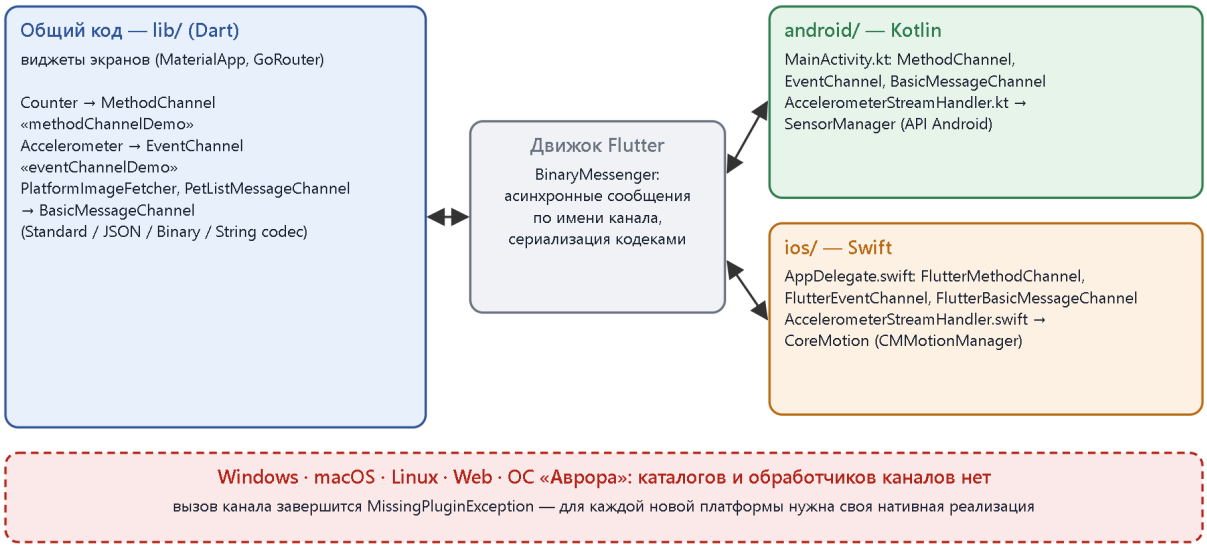


Рисунок 6 — Граница общего и платформенного кода в проекте platform_channels

Доля общего кода посчитана скриптом `tools/code_share.py` по непустым строкам (таблица 3).

Таблица 3 — Распределение кода проекта по областям

Область	Файлов	Непустых строк	Доля исходного кода
`lib/` — общий код (Dart)	10	631	74,2 %
`android/` — код (Kotlin)	2	109	12,8 %
`ios/` — код (Swift)	4	110	12,9 %
`test/` — тесты (Dart, общие)	6	270	—
`android/` — сборка и конфигурация	13	231	—
`ios/` — проект Xcode и конфигурация	20	1091	—

Вывод.

Flutter относится к категории кросс-платформенных мобильных фреймворков из таблицы 1 и обеспечивает переносимость механизмом собственной отрисовки: интерфейс рисует движок Impeller, а код на Dart в релизной сборке компилируется заранее (AOT) в машинный код каждой платформы. Поэтому весь интерфейс и логика примера находятся в общем каталоге `lib/`. Общий код составляет 74,2 % исходного кода (631 из 850 строк), а вместе с тестами — 80,4 %. Платформозависимая часть — это обработчики каналов на Kotlin и Swift (по ~110 строк) и сборочные файлы Gradle и Xcode. Граница проходит по каналам платформы: общий код знает только имя канала и формат сообщения, а реализацию для каждой ОС пишут отдельно на её родном языке. Таких мест в примере четыре. Условных проверок ОС в общем коде нет, поэтому его можно тестировать без устройства, подменяя каналы. Главная сложность переноса на новую платформу — эти обработчики. Для Windows их пишут на C++, для Linux — на C (GTK), для macOS — на Swift, для веба — на Dart с обращением к API браузера через JS-interop [8]; без них вызов канала завершится исключением `MissingPluginException`. Для Astra Linux нужен десктопный вариант Flutter (Linux, GTK) и решение, что делать с акселерометром, которого на ПК обычно нет. Для ОС «Аврора» Flutter поддерживается портом компании «Открытая мобильная платформа»: плагины берутся из её репозиториев, а не с pub.dev, и обработчики каналов пришлось бы реализовать на C++ [12]. Таким образом, стоимость переноса определяется объёмом нативного кода за каналами, а не объёмом общего кода.

Сообщение для обсуждения в группе (2–3 минуты).

Я разбирал официальный пример Flutter `platform_channels`. Рекомендованного в указаниях `provider_shopper` в репозитории flutter/samples уже нет, поэтому я выбрал пример, который как раз показывает границу между общим и платформенным кодом. В корне проекта — `pubspec.yaml` с зависимостями и общий каталог `lib/`: в нём интерфейс на

виджетах и четыре небольших класса-обёртки над каналами платформы. Каналы трёх видов: `MethodChannel` — вызов метода с ответом, `EventChannel` — поток событий от датчика, `BasicMessageChannel` — обмен сообщениями через кодеки JSON, двоичный и строковый. Каталоги `android/` и `ios/` — это нативные проекты: Gradle и Kotlin с одной стороны, Xcode и Swift с другой. В `MainActivity.kt` и `AppDelegate.swift` зарегистрированы обработчики тех же каналов по тем же именам; акселерометр читается через `SensorManager` и CoreMotion. Общий код составляет около трёх четвертей исходников и ни разу не проверяет, на какой ОС выполняется, — различия спрятаны за каналами. Отсюда вывод: добавить новую платформу, например Linux или ОС «Аврора», — значит написать для неё обработчики каналов, а интерфейс и логика переедут без изменений.

4. Задание 4. Сравнительный анализ (вариант 7)

Сценарий: контентное приложение новостного портала для iOS, Android и веба; приоритет — скорость разработки. Сравниваются React Native и Ionic + Capacitor (таблица 4). Сведения взяты из официальной документации и релизов; версии — из `data/tool_versions.json`.

Таблица 4 — Сравнение React Native и Ionic + Capacitor для новостного портала

Критерий	React Native (+ Expo)	Ionic + Capacitor
Целевые платформы и покрытие	iOS и Android — основные; веб, Windows, macOS — внешние платформы (react-native-web, проекты Microsoft) [5]. Astra Linux, РЕД ОС: десктопной поддержки нет. ОС «Аврора»: в документации для разработчиков не упоминается [12]	iOS, Android и веб (PWA) из одной кодовой базы [6, 7]. Astra Linux, РЕД ОС — веб-версия в браузере. ОС «Аврора»: документация описывает запуск веб-приложений (PWA) через фреймворк WebView [12]
Язык и порог входа для команды	JavaScript / TypeScript и React; для нативных модулей нужны Xcode и Gradle. Expo упрощает старт и сборку	JavaScript / TypeScript, HTML, CSS и любой из фреймворков Angular, React, Vue [6]; веб-команда переиспользует навыки и компоненты сайта

Критерий	React Native (+ Expo)	Ionic + Capacitor
Механизм рендеринга и производительность	нативные компоненты ОС: React Native создаёт соответствующие виды Android и iOS [5]; новая архитектура, Hermes — производительность близка к нативной	веб-компоненты в системном WebView и нативные плагины Capacitor [7]; для лент, текстов и изображений производительности достаточно, сложная анимация уступает нативной
Размер дистрибутива и ресурсы	в сборку входят среда React Native и движок Hermes	нативная оболочка и веб-ресурсы, отрисовкой занимается системный WebView; веб-версия не требует установки
Зрелость экосистемы, динамика развития	Meta; последний релиз 0.87.1 от 26.08.2026, новая минорная версия выходит примерно раз в два месяца [9]; Expo SDK 57 (24.09.2026); крупное сообщество, пакеты npm	Ionic (OutSystems); Ionic Framework 9.0.5 (23.09.2026), Capacitor 8.5.2 (11.09.2026); плагины Capacitor и совместимость с плагинами Cordova [7]
Лицензия и стоимость владения	MIT, бесплатно; облачная сборка Expo EAS — по тарифам сервиса; отдельные знания для веб-версии	MIT, бесплатно; одна веб-команда и одна кодовая база для сайта и приложений — ниже стоимость владения
Каналы распространения	Google Play, App Store, RuStore (APK / AAB) [11]	Google Play, App Store, RuStore (APK / AAB) [11], а также веб / PWA без магазинов

Вывод (251 слов).

Для контентного приложения новостного портала выбираю Ionic + Capacitor. Приоритет сценария — скорость разработки для трёх платформ: iOS, Android и веба.

Первый аргумент — веб обязателен, а у Ionic он основная платформа. Компоненты Ionic — стандартные Web Components, из одной кодовой базы собираются мобильные приложения и PWA [6]. Capacitor описан разработчиками как кросс-платформенная нативная среда выполнения для веб-приложений на iOS и Android [7]. React Native создаёт нативные виды Android и iOS [5], а веб поддерживается внешним проектом, поэтому сайт фактически пришлось бы делать и тестировать отдельно.

Второй аргумент — скорость и стоимость. Веб-команде достаточно HTML, CSS и знакомого фреймворка, можно переиспользовать вёрстку и компоненты сайта портала. React Native требует знания React и собственных

компонентов React Native, а при подключении нативных модулей — работы с Xcode и Gradle.

Третий аргумент — производительности WebView хватает: новостное приложение — это ленты, тексты, изображения и видео. Преимущество React Native в нативной отрисовке важно для сложных анимаций и насыщенного интерфейса, а здесь оно не окупает двух кодовых баз.

Четвёртый аргумент — охват отечественных платформ. Документация ОС «Аврора» описывает запуск веб-приложений через WebView, а React Native в ней не упоминается [12]; на Astra Linux и РЕД ОС веб-версия работает в браузере. Мобильные сборки Capacitor — обычные APK / AAB и IPA, поэтому каналы те же: Google Play, App Store и RuStore, который принимает APK и AAB размером до 5 ГБ [11].

Оба инструмента бесплатны (MIT) и активно развиваются: Capacitor 8.5.2 вышел 11.09.2026, React Native 0.87.1 — 26.08.2026. React Native стоит выбрать, если приоритетом станет «нативное» поведение сложного интерфейса или в команде уже есть опыт React Native.

5. Ответы на контрольные вопросы

1. Дайте определение кросс-платформенного программного обеспечения. Перечислите и охарактеризуйте уровни совместимости программ.

Кросс-платформенное (переносимое) ПО работает более чем на одной аппаратной или программной платформе: под разными ОС (Windows, GNU/Linux, macOS, iOS, Android) или на разных архитектурах (x86-64, ARM64).

Уровни совместимости: машинного кода — исполняемый файл запускается без перекомпиляции при одинаковых системе команд и ABI либо через эмуляцию и трансляцию (QEMU, Rosetta 2, WINE); байт-кода — программа компилируется в код виртуальной машины (JVM, CLR, WebAssembly), которая есть на каждой платформе; исходного кода — переносимый язык и стандарты (C/C++, POSIX) плюс кросс-платформенная сборка (CMake) и перекомпиляция под каждую платформу; API — приложение опирается на библиотеку или фреймворк (Qt, GTK, Flutter, React Native), скрывающие различия API ОС.

2. Чем байт-код JVM отличается от WebAssembly как средство обеспечения переносимости?

Байт-код JVM — высокоуровневый объектный байт-код: классы, сборка мусора, загрузчик классов и большая стандартная библиотека Java; исполняется JVM с JIT-компиляцией, изначально для Java и Kotlin, с широким доступом к ОС через стандартную библиотеку.

WebAssembly — низкоуровневый переносимый двоичный формат (стековая машина, линейная память): целевой формат компиляции для C/C++, Rust, Go и других языков. Выполняется в песочнице браузера или автономной средой (Wasmtime), а к системе обращается только через явно импортированные функции (API браузера, WASI). Сборка мусора и другие возможности вошли в спецификацию Wasm 3.0, завершённую 17.09.2025 [13].

3. Как устроен рендеринг во Flutter и почему этому фреймворку не требуются нативные компоненты ОС?

Flutter рисует интерфейс сам: дерево виджетов превращается в дерево объектов отрисовки и слои, а движок Impeller (в вебе — Skia) выводит пиксели на поверхность, которую предоставляет платформа. От ОС нужны только окно или поверхность, события ввода и доступ к сервисам через каналы; элементы управления ОС не используются, поэтому интерфейс одинаков на всех платформах. Impeller — единственный движок на iOS, по умолчанию на Android с API 29, с Flutter 3.47 — и на десктопе [10].

4. Что изменилось в React Native с переходом на «новую архитектуру» (версии 0.76–0.82)?

Асинхронный «мост» с сериализацией в JSON заменён интерфейсом JSI: JavaScript напрямую и синхронно обращается к объектам C++. Появились рендерер Fabric (синхронная раскладка, поддержка конкурентных возможностей React), модули TurboModules (ленивая загрузка через JSI) и генерация типизированных интерфейсов (Codegen). В 0.76 (23.10.2024) новая архитектура включена по умолчанию, в 0.82 (08.10.2025) стала единственной — флаги отключения игнорируются [9]; в 0.84 (11.02.2026) по умолчанию используется Hermes V1.

5. В чём разница между Kotlin Multiplatform и Compose Multiplatform? Что изменил стабильный статус iOS-версии CMP 1.8?

Kotlin Multiplatform — технология общего кода на Kotlin (бизнес-логика, сеть, хранение данных): он компилируется в байт-код JVM, в машинный код через Kotlin/Native (iOS) и в JS/Wasm, а интерфейс на каждой платформе может оставаться нативным.

Compose Multiplatform — декларативный UI-фреймворк на основе Jetpack Compose, работающий поверх KMP на Android, iOS, десктопе и в вебе, то есть общим становится и интерфейс. Стабильный iOS в CMP 1.8 (май 2025) сделал общий интерфейс для Android и iOS пригодным для

промышленной разработки: команда выбирает, делить только логику или логику вместе с UI. Актуальная версия — 1.12.1.

6. Почему приложения Electron имеют большой размер дистрибутива и как эту проблему решает Tauri?

Каждый дистрибутив Electron содержит собственные Chromium (движок отрисовки, V8, графика, ICU, локализации) и Node.js. В практической работе № 3 установщик учебного приложения занял 106 МБ, распакованное приложение — 367 МБ при 12,8 КБ собственного кода. Tauri использует системный WebView (WebView2 в Windows, WKWebView в macOS и iOS, WebKitGTK в Linux, системный WebView в Android) и компактное ядро на Rust, поэтому в дистрибутив попадают только код приложения и небольшой исполняемый файл. Взамен приходится учитывать различия движков WebView между ОС.

7. Какие инструменты подходят для разработки под ОС «Аврора» и почему?

Основной стек — Qt/QML и C++ через «Аврора SDK»: на Qt построен пользовательский интерфейс самой ОС. Поддерживается Flutter в порте компании «Открытая мобильная платформа» (плагины — из её репозиториев, а не с pub.dev), а веб-приложения можно встроить через фреймворк WebView [12]. React Native и Kotlin Multiplatform в документации для разработчиков ОС «Аврора» не упоминаются. Выбор определяется поддержкой производителя ОС: без официального SDK и плагинов доступ к возможностям устройства невозможен.

8. Назовите особенности публикации приложения в RuStore по сравнению с Google Play.

RuStore (VK) принимает пакеты в форматах APK и AAB размером до 5 ГБ, перед публикацией приложение обязательно проходит модерацию [11]; платные приложения публикуют только юридические лица и ИП, зарегистрированные в РФ. Google Play для новых приложений принимает только AAB, берёт единовременный регистрационный взнос и использует

собственную систему платежей, которая для пользователей из России недоступна. Поэтому в RuStore для оплаты используется его собственный платёжный SDK, а российские разработчики обычно публикуются в обоих магазинах.

9. В каких случаях нативная разработка предпочтительнее кросс-платформенной? Приведите два примера.

Когда нужны предельная производительность, прямой доступ к новым API ОС сразу после их выхода, минимальные размер и потребление ресурсов или жёсткие требования безопасности и сертификации. Примеры: 1) мобильный банк с высокими требованиями безопасности и «нативным» поведением (защищённое хранилище ключей, антифрод, виджеты, часы); 2) приложение обработки видео или дополненной реальности, игра — нужны прямой доступ к камере и GPU и минимальные задержки.

10. Какую роль играют CMake и стандарт POSIX в обеспечении переносимости на уровне исходного кода?

POSIX стандартизирует интерфейс прикладной программы к ОС (файлы, процессы, потоки, сигналы, время), поэтому один исходный код на C собирается на всех POSIX-совместимых системах. CMake — метасистема сборки: единое платформенно-независимое описание проекта (`CMakeLists.txt`) порождает родные файлы сборки (Makefile, Ninja, Visual Studio, Xcode), находит компиляторы и библиотеки и позволяет учесть различия платформ (`if(WIN32)`, `find_package`). Вместе они обеспечивают переносимость исходного текста.

11. Почему Apache Cordova сегодня не рекомендуется для новых проектов и что пришло ей на смену?

Cordova — устаревшая гибридная платформа: нативный проект генерируется заново при сборке и фактически не редактируется, экосистема плагинов стареет, поддержка новых версий iOS и Android отстаёт. На смену пришёл Capacitor от команды Ionic — «следующая эволюция гибридных приложений» [7]: нативные проекты iOS и Android — полноценная часть

исходников (открываются в Xcode и Android Studio), современный API плагинов, поддержка PWA и совместимость с плагинами Cordova для миграции.

12. Как ИИ-ассистенты разработчика влияют на выбор инструментария в 2025–2026 годах? Приведите аргументы «за» и «против» их использования в учебном проекте.

Ассистенты лучше помогают с популярными инструментами, для которых много открытого кода (React, Flutter, Electron), и хуже — с новыми и нишевыми («Аврора SDK», свежие API). Это смещает выбор в пользу распространённых стеков. «За»: быстрее пишется шаблонный код и тесты, проще разобраться в ошибке и перенести код между фреймворками. «Против»: ассистент уверенно предлагает устаревшие версии и API (в этой работе версии из учебников и указаний уже разошлись с первоисточниками), небезопасные настройки (например, `nodeIntegration: true` в Electron) и снижает понимание архитектуры. В учебном проекте ассистент допустим для рутины при условии, что каждый факт и версия проверяются по документации, а код обучающийся понимает и может защитить.

Заключение

В работе составлена классификация 16 инструментов кросс-платформенной разработки по семи категориям с актуальными версиями, проверенными по первоисточникам на дату выполнения. Подготовлено рабочее окружение и Git-репозиторий с циклом clone → commit → push. Разобрано устройство официального примера Flutter `platform_channels`: около 74 % исходного кода общие, платформенная часть сосредоточена в обработчиках каналов на Kotlin и Swift. Для новостного портала (вариант 7) обоснован выбор Ionic + Capacitor. Главный практический вывод: сведения об инструментах быстро устаревают, поэтому их нужно проверять по официальным релизам и документации.

Список использованных источников

1) Рочев К. В. Разработка мобильных и кроссплатформенных приложений : учебник. — М. : Русайнс, 2025. — 247 с.

2) Беспалов Д. А., Гушанский С. М., Коробейникова Н. М. Операционные системы реального времени и технологии разработки кроссплатформенного программного обеспечения. Ч. 1. — Ростов-на-Дону ; Таганрог : Изд-во ЮФУ, 2019. — 139 с.

3) GitHub REST API: Releases [Электронный ресурс]. — URL: https://docs.github.com/rest/releases (дата обращения: 27.09.2026).

4) flutter/samples: platform_channels [Электронный ресурс]. — URL: https://github.com/flutter/samples/tree/main/platform_channels (дата обращения: 27.09.2026).

5) React Native. Core Components and Native Components [Электронный ресурс]. — URL: https://reactnative.dev/docs/intro-react-native-components (дата обращения: 27.09.2026).

6) Ionic Framework. Documentation [Электронный ресурс]. — URL: https://ionicframework.com/docs (дата обращения: 27.09.2026).

7) Capacitor. Documentation [Электронный ресурс]. — URL: https://capacitorjs.com/docs (дата обращения: 27.09.2026).

8) Flutter. Platform channels [Электронный ресурс]. — URL: https://docs.flutter.dev/platform-integration/platform-channels (дата обращения: 27.09.2026).

9) React Native. Blog (релизы 0.76–0.87) [Электронный ресурс]. — URL: https://reactnative.dev/blog (дата обращения: 27.09.2026).

10) Flutter. Impeller rendering engine [Электронный ресурс]. — URL: https://docs.flutter.dev/perf/impeller (дата обращения: 27.09.2026).

11) RuStore. Публикация приложений [Электронный ресурс]. — URL: https://www.rustore.ru/help/developers/publishing-and-verifying-apps/app-publication (дата обращения: 27.09.2026).

12) ОС Аврора. Разработка ПО; Flutter для ОС Аврора [Электронный ресурс]. — URL: https://developer.auroraos.ru/doc/software_development (дата обращения: 27.09.2026).

13) WebAssembly. News: Wasm 3.0 Completed [Электронный ресурс]. — URL: https://webassembly.org/news/ (дата обращения: 27.09.2026).

14) Релизы и витрины инструментов таблицы 1: https://docs.flutter.dev/release/archive, https://dotnet.microsoft.com/download/dotnet, https://download.qt.io/official_releases/qt/6.11/, https://github.com/AvaloniaUI/Avalonia/releases/tag/12.1.3, https://github.com/JetBrains/compose-multiplatform/releases/tag/v1.12.1, https://github.com/JetBrains/kotlin/releases/tag/v2.4.20, https://github.com/Kitware/CMake/releases/tag/v4.4.3, https://github.com/bytecodealliance/wasmtime/releases/tag/v49.0.1, https://github.com/dotnet/maui/releases/tag/10.0.110, https://github.com/electron/electron/releases/tag/v44.4.5, https://github.com/ionic-team/capacitor/releases/tag/8.5.2, https://github.com/ionic-team/ionic-framework/releases/tag/v9.0.5, https://github.com/microsoft/WSL/releases/tag/2.7.14, https://github.com/moby/moby/releases/tag/docker-v29.8.1, https://github.com/react/react-native/releases/tag/v0.87.1, https://github.com/tauri-apps/tauri/releases/tag/tauri-v2.12.0, https://nodejs.org/dist/index.json, https://www.npmjs.com/package/expo, https://www.python.org/downloads/ (дата обращения: 27.09.2026).

15) Витрины приложений: https://flutter.dev/showcase, https://reactnative.dev/showcase, https://kotlinlang.org/case-studies, https://www.electronjs.org/apps, https://avaloniaui.net/showcase, https://ionic.io/customers, https://github.com/gitbutlerapp/gitbutler (дата обращения: 27.09.2026).